Homework: Attention Mechanism and Transformer

Complete Solutions (Exam Answer Format)

Contents

1 Q1. Limitations of RNNs and the Attention Mechanism **3**

1.1 Part 1: Limitations of conventional RNNs . . . 3

1.2 Part 2: How attention addresses these limitations . . . 3

1.3 Part 3: Numerical . . . 4

2 Q2. Self-Attention and Scaled Dot-Product Attention **5**

2.1 Part 1: Self-attention . . . 5

2.2 Part 2: Derivation of scaled dot-product attention . . . 5

2.3 Part 3: Numerical . . . 6

3 Q3. Transformer Architecture and Multi-Head Attention Dimensions **8**

3.1 Part 1: Transformer architecture (Vaswani et al., 2017) . . . 8

3.2 Part 2: Numerical . . . 9

4 Q4. Positional Encoding **11**

4.1 Part 1: Need for positional encoding . . . 11

4.2 Part 2: Combining positional encoding with token embeddings . . . 11

4.3 Part 3: Numerical . . . 11

4.4 Part 4: Fixed vs learned positional encodings . . . 12

5 Q5. Sparse Attention for Long Sequences **13**

5.1 Part 1: How sparse attention reduces connections . . . 13

5.2 (a) Why dense attention is inefficient for long sequences . . . 13

5.3 (b) Proposed sparse attention pattern: local (sliding window) + global attention . . . 13

5.4 (c) How the design captures nearby and distant context . . . 14

5.5 (d) Limitation of the proposed design . . . 14

6 Q6. Masked Self-Attention **16**

6.1 Part 1: Masked (causal) self-attention . . . 16

6.2 Part 2: Application to “The student is reading a book.” . . . 16

7 Q7. BERT vs GPT **18**

7.1 Part 1: Architecture . . . 18

7.2 Part 2: Training objectives . . . 18

7.3 Part 3: Why BERT suits understanding and GPT suits generation . . . 19

7.4 Part 4: Applications . . . 19

8 Q8. Vision Transformer (ViT) 20

8.1 Part 1: Basic architecture of ViT (Dosovitskiy et al., 2020) 20

8.2 Part 2: Converting image patches into tokens 20

8.3 Part 3: Numerical – number of patches 21

8.4 Part 4: Role of positional information and the encoder 21

9 Q9. Embedding and Head Dimensions for a Sentence 22

10 Q10. Dimensions in an Encoder–Decoder Transformer 24

Q1. Limitations of RNNs and the Attention Mechanism

Discuss two major limitations of conventional RNNs when processing long sequences. Explain how attention addresses them. Encoder hidden states: $h_1 = [1, 2]$, $h_2 = [2, 1]$, $h_3 = [3, 2]$. Attention weights: $\alpha_{t,1} = 0.2$, $\alpha_{t,2} = 0.5$, $\alpha_{t,3} = 0.3$. (a) Verify normalization. (b) Weighted contribution of each hidden state. (c) Context vector c_t. (d) Which hidden state contributes most?

Part 1: Limitations of conventional RNNs

An RNN processes a sequence one token at a time, updating a hidden state $h_t = f(W_{hh}h_{t-1} + W_{xh}x_t + b)$. In the classic encoder–decoder (seq2seq) model, the encoder's last hidden state h_T is the *only* summary of the whole input passed to the decoder.

Limitation 1 – Fixed-length context bottleneck (long-range dependency problem):

- The entire input sentence, of any length, must be compressed into one fixed-size vector h_T.
- For long sentences, information about early tokens is overwritten by later tokens, so the decoder "forgets" the beginning. Translation quality of basic seq2seq models drops sharply as sentence length increases.

Limitation 2 – Vanishing and exploding gradients:

- In back-propagation through time, the gradient from step T to step k contains a product of Jacobians:

$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}}, \qquad \frac{\partial h_t}{\partial h_{t-1}} = \operatorname{diag}(f'(\cdot))\, W_{hh}.$$

- If the norms of these Jacobians are less than 1 the product shrinks exponentially with $T - k$ (vanishing gradients); if greater than 1 it grows exponentially (exploding gradients). Hence the network cannot learn dependencies between distant tokens. LSTM/GRU gates reduce but do not eliminate this.

Additional limitation – Sequential computation: h_t depends on h_{t-1}, so time steps cannot be computed in parallel. Training on long sequences is slow and does not use GPUs efficiently. The path length between two tokens i and j is $O(|i - j|)$.

Part 2: How attention addresses these limitations

Instead of using only h_T, the decoder at each output step t looks at *all* encoder hidden states $h_1, \ldots, h_T$ and builds its own context vector (Bahdanau et al., 2015):

1. **Alignment scores:** $e_{t,i} = \text{score}(s_{t-1}, h_i)$, e.g. additive $v^\top \tanh(W_s s_{t-1} + W_h h_i)$ or dot-product $s_{t-1}^\top h_i$, where s_{t-1} is the previous decoder state.
2. **Attention weights (softmax):** $\alpha_{t,i} = \dfrac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$, so $\alpha_{t,i} \geq 0$ and $\sum_i \alpha_{t,i} = 1$.
3. **Context vector:** $c_t = \sum_{i=1}^{T} \alpha_{t,i}\, h_i$.
4. **Decoding:** $s_t = f(s_{t-1}, y_{t-1}, c_t)$ and the output word is predicted from s_t and c_t.

Why this solves the problems:

- **No bottleneck:** A different context vector is computed for every output step, focusing on the relevant input tokens, so information from any position is available regardless of sentence length.

- **Short gradient paths:** Each output is directly connected to every encoder state through the weights $\alpha_{t,i}$, so gradients flow directly to distant tokens (path length $O(1)$), reducing the vanishing-gradient problem.
- **Interpretability:** The weights $\alpha_{t,i}$ show which input words the model used (soft alignment).
- **Parallelism:** Taken further, self-attention (Transformer) removes recurrence entirely so all positions are processed in parallel.

Part 3: Numerical

Given: $h_1 = [1, 2]$, $h_2 = [2, 1]$, $h_3 = [3, 2]$; $\alpha_{t,1} = 0.2$, $\alpha_{t,2} = 0.5$, $\alpha_{t,3} = 0.3$.
Formula: $c_t = \sum_{i=1}^{3} \alpha_{t,i}\, h_i$.

(a) Normalization property

Attention weights are produced by a softmax, so they must satisfy (i) $0 \le \alpha_{t,i} \le 1$ and (ii) $\sum_i \alpha_{t,i} = 1$.

$$\text{(i)} \quad 0 \le 0.2,\ 0.5,\ 0.3 \le 1 \quad \checkmark$$

$$\text{(ii)} \quad \sum_{i=1}^{3} \alpha_{t,i} = 0.2 + 0.5 + 0.3 = 1.0 \quad \checkmark$$

The weights are non-negative and sum to 1, so the normalization property is satisfied.

(b) Weighted contribution of each encoder hidden state

$$\alpha_{t,1} h_1 = 0.2 \times [1, 2] = [0.2 \times 1,\ 0.2 \times 2] = [0.2,\ 0.4]$$
$$\alpha_{t,2} h_2 = 0.5 \times [2, 1] = [0.5 \times 2,\ 0.5 \times 1] = [1.0,\ 0.5]$$
$$\alpha_{t,3} h_3 = 0.3 \times [3, 2] = [0.3 \times 3,\ 0.3 \times 2] = [0.9,\ 0.6]$$

$\alpha_{t,1} h_1 = [0.2, 0.4], \quad \alpha_{t,2} h_2 = [1.0, 0.5], \quad \alpha_{t,3} h_3 = [0.9, 0.6]$

(c) Context vector

$$\begin{aligned} c_t &= [0.2, 0.4] + [1.0, 0.5] + [0.9, 0.6] \\ &= [0.2 + 1.0 + 0.9,\ \ 0.4 + 0.5 + 0.6] \\ &= [2.1,\ 1.5] \end{aligned}$$

$c_t = [2.1,\ 1.5]$

(d) Hidden state contributing most

Compare the attention weights and the size (Euclidean norm) of each weighted contribution:

State	Weight $\alpha_{t,i}$	Contribution $\alpha_{t,i} h_i$	Norm $\lVert \alpha_{t,i} h_i \rVert$
h_1	0.2	$[0.2, 0.4]$	$\sqrt{0.04 + 0.16} = \sqrt{0.20} = 0.447$
h_2	**0.5**	$[1.0, 0.5]$	$\sqrt{1.00 + 0.25} = \sqrt{1.25} = \mathbf{1.118}$
h_3	0.3	$[0.9, 0.6]$	$\sqrt{0.81 + 0.36} = \sqrt{1.17} = 1.082$

h_2 contributes most: it has the largest attention weight ($\alpha_{t,2} = 0.5$, i.e. 50% of the attention) and the largest weighted contribution ($\lVert [1.0, 0.5] \rVert = 1.118$). The decoder at this step is focusing mainly on the second input token.

Note: Although $h_3 = [3, 2]$ is the largest vector, its lower weight (0.3) makes its contribution slightly smaller, which shows that the attention weight – not the raw size of the hidden state – decides the focus.

Q2. Self-Attention and Scaled Dot-Product Attention

Define self-attention and explain how it captures long-range relationships. Explain the roles of Q, K, V. Derive scaled dot-product attention and explain the scaling factor. $Q = [1, 0]$, $K_1 = [1, 0]$, $K_2 = [0, 1]$, $V_1 = [2, 1]$, $V_2 = [0, 3]$. (a) Dot products. (b) Scaled scores with $d_k = 2$. (c) Softmax weights. (d) Attention output.

Part 1: Self-attention

Definition: Self-attention (intra-attention) is an attention mechanism in which every token of a sequence attends to every token *of the same sequence* (including itself), and its new representation is computed as a weighted sum of the representations of all tokens, with weights given by their relevance.

Example: in "The animal didn't cross the street because *it* was too tired", self-attention lets the token "it" give a high weight to "animal", so the representation of "it" contains information about what it refers to.

How it captures long-range relationships:

- Every pair of tokens is connected directly: the attention score between positions i and j is computed in one step regardless of the distance $|i - j|$. The maximum path length between any two tokens is $O(1)$, compared with $O(n)$ in an RNN.
- Information is not passed through a chain of hidden states, so it does not fade with distance and gradients do not vanish along long paths.
- The weights are *dynamic*: they are computed from the content of the tokens, so the model decides which words are related for each input (syntax, coreference, semantics).
- All positions are processed in parallel as matrix multiplications, so long sequences are handled efficiently on GPUs (up to the $O(n^2)$ cost).

Roles of Query, Key and Value: For input embeddings $X \in \mathbb{R}^{n \times d_{\text{model}}}$, three learned projection matrices give

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V, \qquad W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k},\ W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

- **Query (q_i):** "What am I looking for?" – represents the current token asking for relevant information.
- **Key (k_j):** "What do I contain / how can I be matched?" – every token's label that is compared with the query. The match $q_i \cdot k_j$ decides how much token i attends to token j.
- **Value (v_j):** "What information do I give?" – the actual content that is passed on and mixed according to the attention weights.

Analogy: information retrieval – a search query is matched against keys (titles) and the contents (values) of the best-matching items are returned. Separate projections let the same token play different roles (what it seeks vs. what it offers).

Part 2: Derivation of scaled dot-product attention

Step 1 – Similarity scores. The relevance of token j to token i is measured by the dot product of query and key: $e_{ij} = q_i \cdot k_j = \sum_{m=1}^{d_k} q_{im} k_{jm}$. For all pairs at once: $E = QK^\top \in \mathbb{R}^{n \times n}$.

Step 2 – Scaling. Divide by $\sqrt{d_k}$: $S = \dfrac{QK^\top}{\sqrt{d_k}}$.

Step 3 – Normalization with softmax (row-wise). $\alpha_{ij} = \dfrac{\exp(s_{ij})}{\sum_{l=1}^{n} \exp(s_{il})}$, so each row of weights is non-negative and sums to 1.

Step 4 – Weighted sum of values. $z_i = \sum_{j=1}^{n} \alpha_{ij} v_j$. In matrix form:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V}$$

with output dimension $n \times d_v$.

Purpose of the scaling factor $\sqrt{d_k}$:

Assume the components of q and k are independent random variables with mean 0 and variance 1. Then

$$\mathbb{E}[q \cdot k] = \sum_{m=1}^{d_k} \mathbb{E}[q_m]\,\mathbb{E}[k_m] = 0,$$
$$\text{Var}(q \cdot k) = \sum_{m=1}^{d_k} \text{Var}(q_m k_m) = \sum_{m=1}^{d_k} \mathbb{E}[q_m^2]\,\mathbb{E}[k_m^2] = \sum_{m=1}^{d_k} 1 \cdot 1 = d_k.$$

So the standard deviation of the dot product grows as $\sqrt{d_k}$ (e.g. $d_k = 64 \Rightarrow \text{std} = 8$). Dividing by $\sqrt{d_k}$ gives

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1.$$

- Without scaling, large dot products push the softmax into *saturation*: one weight becomes almost 1 and the rest almost 0.
- The softmax gradient is $\dfrac{\partial \alpha_i}{\partial s_j} = \alpha_i(\delta_{ij} - \alpha_j)$. When $\alpha_i \approx 1$ or ≈ 0, this is ≈ 0, so gradients vanish and learning becomes very slow.
- Scaling keeps the scores in a range where the softmax is smooth, giving stable gradients and allowing attention to be spread over several tokens.

Part 3: Numerical

Given: $Q = [1, 0]$, $K_1 = [1, 0]$, $K_2 = [0, 1]$, $V_1 = [2, 1]$, $V_2 = [0, 3]$, $d_k = 2$.

(a) Dot products

$$Q \cdot K_1 = (1)(1) + (0)(0) = 1 + 0 = 1$$
$$Q \cdot K_2 = (1)(0) + (0)(1) = 0 + 0 = 0$$

$$Q \cdot K_1 = 1, \quad Q \cdot K_2 = 0$$

(b) Scaled attention scores ($\sqrt{d_k} = \sqrt{2} = 1.4142$)

$$s_1 = \frac{Q \cdot K_1}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} = \frac{1}{1.4142} = 0.7071$$
$$s_2 = \frac{Q \cdot K_2}{\sqrt{d_k}} = \frac{0}{\sqrt{2}} = 0$$

$$s_1 = 0.7071, \quad s_2 = 0$$

(c) Softmax attention weights

$$
\begin{aligned}
e^{s_1} = e^{0.7071} = 2.0281, \qquad & e^{s_2} = e^0 = 1 \\
\text{Sum} = 2.0281 + 1 = 3.0281 & \\
\alpha_1 = \frac{2.0281}{3.0281} = 0.6698, \qquad & \alpha_2 = \frac{1}{3.0281} = 0.3302 \\
\text{Check: } \alpha_1 + \alpha_2 = 0.6698 + 0.3302 = 1.0000 \ \checkmark &
\end{aligned}
$$

$\alpha_1 \approx 0.670, \quad \alpha_2 \approx 0.330$

(d) Attention output

$$
\begin{aligned}
z &= \alpha_1 V_1 + \alpha_2 V_2 \\
&= 0.6698 \times [2, 1] + 0.3302 \times [0, 3] \\
&= [1.3395,\ 0.6698] + [0,\ 0.9907] \\
&= [1.3395 + 0,\quad 0.6698 + 0.9907] \\
&= [1.3395,\ 1.6605]
\end{aligned}
$$

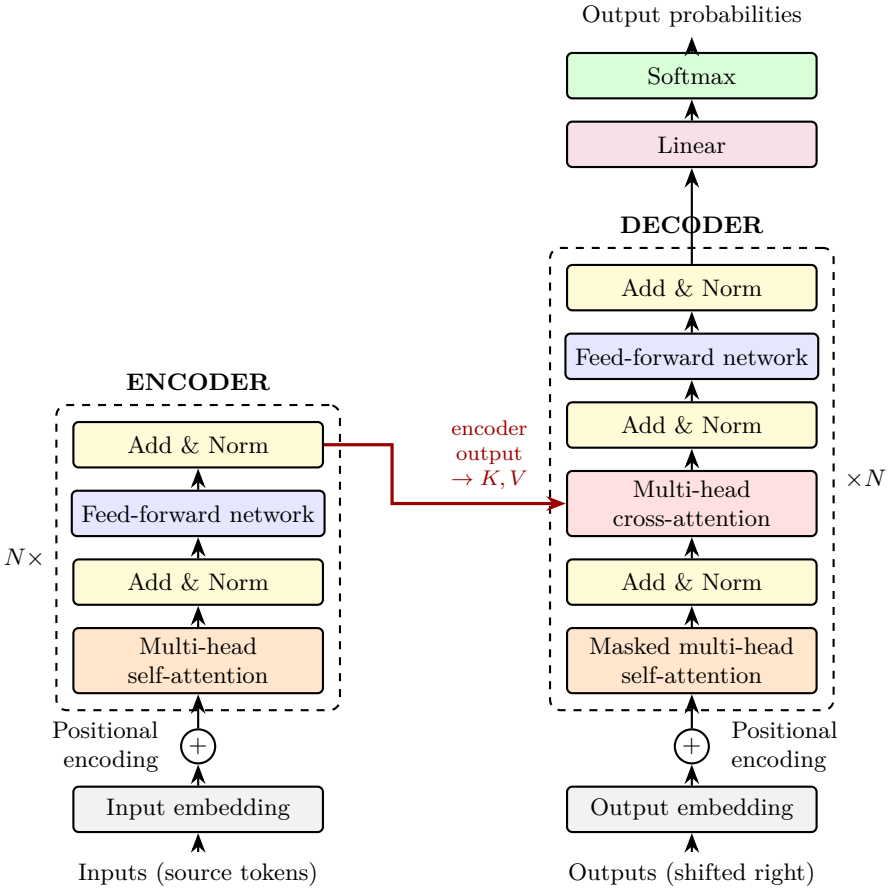
Attention output $z \approx [1.34,\ 1.66]$

Interpretation: The query $[1, 0]$ is identical in direction to K_1 and orthogonal to K_2, so it gives more weight (67%) to V_1. Because of the softmax, V_2 still receives a non-zero weight (33%); the output is a soft mixture, not a hard selection.

Q3. Transformer Architecture and Multi-Head Attention Dimensions

Explain the major components of the Transformer with a block diagram. Explain self-attention, cross-attention and feed-forward networks. Hidden dimension $d_{\text{model}} = 512$, $h = 8$ heads, sequence length $n = 10$. (a) Dimension of Q, K, V vectors per head. (b) Dimensions of Q, K, V matrices for one head. (c) Attention score matrix for one head. (d) Total attention scores across 8 heads. (e) Why multi-head attention is better than one head.

Part 1: Transformer architecture (Vaswani et al., 2017)



(Each "Add & Norm" block adds a residual connection from the input of the sub-layer below it.)

Major components:

1. **Input / output embeddings:** Convert token IDs to vectors of dimension d_{model} (512); multiplied by $\sqrt{d_{\text{model}}}$.
2. **Positional encoding:** Added to embeddings to give word-order information (see Q4).
3. **Encoder:** A stack of $N = 6$ identical layers. Each layer has (i) a multi-head self-attention sub-layer and (ii) a position-wise feed-forward sub-layer. Each sub-layer is wrapped as LayerNorm(x+ Sublayer(x)) (residual connection + layer normalization). Output: contextual representations of all source tokens.
4. **Decoder:** A stack of $N = 6$ layers, each with three sub-layers: (i) masked multi-head self-attention, (ii) multi-head cross-attention (encoder–decoder attention), (iii) feed-forward network; each with residual + LayerNorm.
5. **Multi-head attention:** $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$, where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

6. **Residual connections and Layer Normalization:** Help gradient flow through deep stacks and stabilize training.
7. **Final linear + softmax:** Projects decoder output to vocabulary size and gives the probability of the next token.

Functions of the three sub-layer types:

- **Self-attention:** Q, K, V all come from the same sequence. Each token gathers information from all other tokens of its own sequence, building context-aware representations (e.g. resolving "it" to its noun). In the encoder it is bidirectional; in the decoder it is *masked* so a position cannot see future tokens.
- **Cross-attention (encoder–decoder attention):** Queries come from the decoder; keys and values come from the encoder output. It lets each target position look at the relevant source tokens – this is where the alignment between source and target languages is learned in translation.
- **Feed-forward network (FFN):** Applied to each position separately and identically: $\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$, with $W_1 \in \mathbb{R}^{512\times2048}$, $W_2 \in \mathbb{R}^{2048\times512}$. Attention mixes information *across* tokens; the FFN transforms each token's features non-linearly, adding representational capacity (and storing much of the model's knowledge).

Part 2: Numerical

Given: $d_{\text{model}} = 512$, $h = 8$, $n = 10$.
Formula: $d_k = d_v = \dfrac{d_{\text{model}}}{h}$.

(a) Dimension of Q, K, V vectors per head

$$d_k = d_v = \frac{d_{\text{model}}}{h} = \frac{512}{8} = 64$$

Each query, key and value vector in one head has dimension 64.

(b) Dimensions of Q, K, V matrices for one head

Each head projects the input $X \in \mathbb{R}^{10\times512}$ with $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{512\times64}$:

$$Q_i = XW_i^Q : \ (10 \times 512)(512 \times 64) = 10 \times 64$$

and similarly for K_i and V_i.

$Q_i \in \mathbb{R}^{10\times64}$, $K_i \in \mathbb{R}^{10\times64}$, $V_i \in \mathbb{R}^{10\times64}$ (one row of size 64 per token).

(c) Attention score matrix for one head

$$Q_iK_i^\top : \ (10 \times 64)(64 \times 10) = 10 \times 10$$

Attention score matrix $\in \mathbb{R}^{10\times10}$ (= 100 scores: one for every query–key pair). After softmax, multiplying by V_i gives $(10 \times 10)(10 \times 64) = 10 \times 64$.

(d) Total attention score values across 8 heads

$$\text{Total} = h \times n \times n = 8 \times 10 \times 10 = 800$$

800 attention score values (8 matrices of size 10×10).

The 8 head outputs (10×64 each) are concatenated to 10×512 and multiplied by $W^O \in \mathbb{R}^{512\times512}$ to give the final 10×512 output.

(e) Why multi-head attention is more effective than a single head

1. **Multiple representation subspaces:** Each head has its own W_i^Q, W_i^K, W_i^V, so it projects tokens into a different 64-dimensional subspace and can learn a different kind of relationship.
2. **Different relations in parallel:** One head may track syntactic dependencies (subject–verb), another coreference ("it" $\rightarrow$ "animal"), another nearby/positional context, another semantic similarity. A single head must combine all of these into one set of weights.
3. **Avoids averaging away information:** A single softmax produces one weighted average per token, which blurs several distinct relations together. With 8 heads, a token can attend strongly to 8 different positions at once.
4. **Same cost:** Because $d_k = d_{\text{model}}/h$, total computation and parameters are about the same as one head of full dimension 512.
5. **Robustness:** Redundancy across heads makes the model less dependent on any single attention pattern; heads also run in parallel.

Q4. Positional Encoding

Why do Transformers need positional encoding, whereas RNNs process tokens sequentially? How is positional encoding combined with token embeddings? $X = [0.2, 0.5, 0.3, 0.1]$, $P = [0.1, 0.2, 0.4, 0.3]$. Find the resulting representation. Differentiate fixed and learned positional encodings.

Part 1: Need for positional encoding

RNNs: Process tokens one after another; h_t depends on h_{t-1}. The order of the tokens is built into the computation, so position information is automatically present.

Transformers: Self-attention processes all tokens in parallel and computes $\text{softmax}(QK^\top/\sqrt{d_k})V$, which treats the input as an unordered *set*.

- Self-attention is **permutation-equivariant**: if the input rows are permuted by a permutation matrix Π, then $\text{Attn}(\Pi X) = \Pi\,\text{Attn}(X)$. Each token gets the same output regardless of where it appears.
- Hence without position information, "The dog bit the man" and "The man bit the dog" give the same set of token representations, although the meanings are different.
- Positional encoding injects the position of each token into its representation so that word order, distance and relative position can be used by attention.

Part 2: Combining positional encoding with token embeddings

For token x_t at position t, the input to the first layer is the **element-wise sum**

$$z_t = \text{Embedding}(x_t) \cdot \sqrt{d_{\text{model}}} \;+\; PE_t, \qquad z_t, PE_t \in \mathbb{R}^{d_{\text{model}}}.$$

For the whole sequence: $Z = X_{\text{emb}} + PE$, with both matrices of shape $n \times d_{\text{model}}$.

Sinusoidal (fixed) encoding used in the original Transformer:

$$PE_{(pos,\,2i)} = \sin\!\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \qquad PE_{(pos,\,2i+1)} = \cos\!\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

where pos is the position and i indexes the dimension pair. Each dimension pair is a sinusoid of a different wavelength (from 2π to $10000 \cdot 2\pi$). Properties: unique pattern for every position, values bounded in $[-1, 1]$, and PE_{pos+k} is a linear function (a rotation) of PE_{pos}, which helps the model learn relative positions.

Addition (rather than concatenation) keeps the dimension at d_{model}, adds no parameters, and the network learns to use the positional and semantic parts of the vector together.

Part 3: Numerical

Given: $X = [0.2, 0.5, 0.3, 0.1]$, $P = [0.1, 0.2, 0.4, 0.3]$ (both 4-dimensional).
Formula: $Z = X + P$ (element-wise).

$$\begin{aligned} Z &= [0.2 + 0.1,\quad 0.5 + 0.2,\quad 0.3 + 0.4,\quad 0.1 + 0.3] \\ &= [0.3,\ 0.7,\ 0.7,\ 0.4] \end{aligned}$$

Resulting representation $Z = [0.3,\ 0.7,\ 0.7,\ 0.4]$ (still 4-dimensional).

Part 4: Fixed vs learned positional encodings

Aspect	Fixed (sinusoidal)	Learned
Definition	Computed by a fixed mathematical formula (sin/cos of different frequencies)	A trainable embedding vector for each position, $E_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d_{\text{model}}}$, learned by back-propagation
Parameters	None (0 trainable parameters)	$L_{\max} \times d_{\text{model}}$ parameters (e.g. $512 \times 768 \approx 0.39$M in BERT)
Sequence length	Can be computed for any position, so it can extrapolate to sequences longer than those seen in training (in principle)	Limited to the maximum length $L_{\max}$ used in training; positions beyond it have no embedding
Relative position	PE_{pos+k} is a linear transformation of PE_{pos}, which encodes relative distance	No built-in structure; relationships must be learned from data
Flexibility	Not adapted to the data or task	Adapts to the data and task; can learn task-specific positional patterns
Data need	Works well even with less data	Needs enough data to learn good position vectors
Used in	Original Transformer (Vaswani et al.)	BERT, GPT-2, ViT
Performance	In the original paper both gave nearly identical results	–

(Modern LLMs often use *relative* schemes such as RoPE or ALiBi, which encode relative distance directly inside attention.)

Q5. Sparse Attention for Long Sequences

Explain how sparse attention reduces the number of token-to-token connections compared with standard self-attention. A Transformer must process long sequences efficiently while capturing important context. (a) Why dense attention is inefficient for long sequences. (b) Design a sparse pattern using local and global attention. (c) How the design captures nearby and distant context. (d) One limitation.

Part 1: How sparse attention reduces connections

In standard (dense) self-attention, each of the n tokens attends to all n tokens, so there are $n \times n = n^2$ connections. Sparse attention allows each token to attend only to a *selected subset* S_i of positions, with $|S_i| \ll n$:

$$\text{Attn}(q_i) = \sum_{j \in S_i} \frac{\exp(q_i \cdot k_j / \sqrt{d_k})}{\sum_{l \in S_i} \exp(q_i \cdot k_l / \sqrt{d_k})} v_j.$$

In practice this is done by setting the scores of non-selected pairs to $-\infty$ (mask $M_{ij} = -\infty$ for $j \notin S_i$), so their softmax weights are 0, and (in efficient implementations) not computing them at all. The number of connections falls from n^2 to $\sum_i |S_i| = O(n \cdot s)$, where $s = |S_i|$ is constant or grows slowly (e.g. $O(n\sqrt{n})$ in the Sparse Transformer, $O(n)$ in Longformer/BigBird).

(a) Why dense attention is inefficient for long sequences

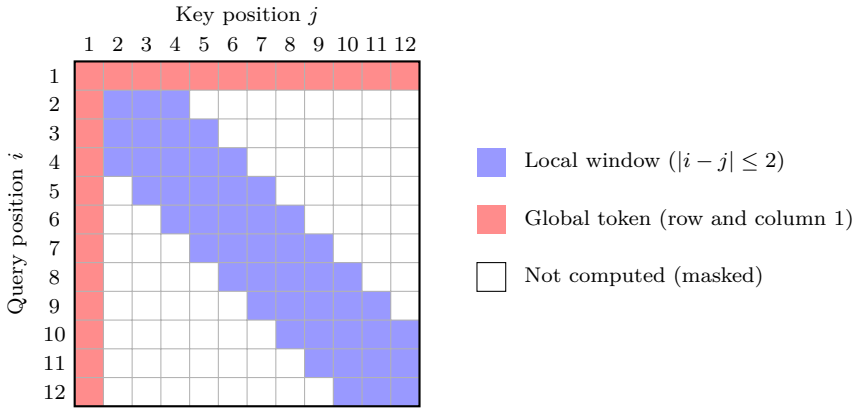
1. **Quadratic time:** Computing $QK^\top$ needs $n^2 d_k$ multiplications per head, i.e. $O(n^2 d)$ per layer. Doubling the length quadruples the cost.
2. **Quadratic memory:** The $n \times n$ score matrix must be stored for every head and layer (and kept for back-propagation). For $n = 4096$: $4096^2 = 16{,}777{,}216$ scores per head per layer; with 12 heads $\times$ 12 layers in FP32 (4 bytes) this is $16{,}777{,}216 \times 144 \times 4 \approx 9.7$ GB just for attention scores.
3. **Wasted computation:** Most attention weights in long documents are close to zero – a token typically depends strongly on nearby tokens and on a few important distant tokens, not on all n.
4. **Practical limit:** Because of this cost, models like BERT are limited to 512 tokens and long documents must be truncated or split, losing context.

(b) Proposed sparse attention pattern: local (sliding window) + global attention

Design (similar to Longformer / BigBird):

1. **Local sliding-window attention:** Each token i attends to the w tokens on each side: $S_i^{\text{local}} = \{\, j : |i - j| \le w \,\}$, giving $2w + 1$ connections per token.
2. **Global attention:** A small set G of g tokens is chosen as global tokens – e.g. the `[CLS]` token, the question tokens in QA, sentence/section-start tokens. Global tokens attend to *all* tokens, and *all* tokens attend to the global tokens (symmetric).
3. **Combined pattern:** For a non-global token: $S_i = \{\, j : |i - j| \le w \,\} \cup G$. For a global token: $S_i = \{1, \ldots, n\}$.
4. **(Optional) Dilated windows / random links** in some heads to widen the receptive field further.

Attention mask illustration ($n = 12$, window $w = 2$, global token = position 1, `[CLS]`); a shaded cell (i, j) means query i attends to key j:



Number of connections (illustration above): Dense: $12 \times 12 = 144$. Sparse: count the shaded cells. Row 1 (global): 12. For rows $i = 2, \ldots, 12$, column 1 is always included (global), plus local cells $j \geq 2$ with $|i-j| \leq 2$: row 2: $j \in \{1,2,3,4\} \to 4$; row 3: $\{1,2,3,4,5\} \to 5$; row 4: $\{1,2,\ldots,6\} \to 6$; rows 5–10: $1+5=6$ each $\to 36$; row 11: $\{1,9,10,11,12\} \to 5$; row 12: $\{1,10,11,12\} \to 4$.
Total $= 12+4+5+6+36+5+4 = 72$, i.e. 50% of the dense connections even for this very short sequence.

For a long sequence ($n = 4096$, $w = 256$, $g = 2$ global tokens), approximately:

$$\begin{aligned} \text{Dense} &= n^2 = 4096^2 = 16{,}777{,}216 \\ \text{Local} &\approx n(2w+1) = 4096 \times 513 = 2{,}101{,}248 \\ \text{Global} &\approx 2gn = 2 \times 2 \times 4096 = 16{,}384 \\ \text{Sparse total} &\approx 2{,}101{,}248 + 16{,}384 = 2{,}117{,}632 \;\approx\; 12.6\% \text{ of dense} \end{aligned}$$

(slight over-count, as edge tokens have fewer neighbours and some cells overlap). The cost is $O(n(w+g))$, i.e. **linear** in n for fixed w and g, instead of quadratic.

(c) How the design captures nearby and distant context

- **Nearby context (local attention):** Most linguistic dependencies (phrases, adjective–noun, subject–verb within a clause) are short-range. The sliding window gives every token full attention to its $2w$ neighbours, capturing these precisely.
- **Growing receptive field through layers:** After L stacked layers, information can spread $L \times w$ positions (like stacked CNN filters). With $w = 256$ and $L = 12$, the receptive field is $12 \times 256 = 3072$ tokens on each side.
- **Distant context (global attention):** Global tokens see the whole sequence and every token sees them. Any two tokens i and j, however far apart, are connected in at most two steps: $i \to g \to j$. Global tokens act as hubs that collect and broadcast document-level information (e.g. the question in QA, the topic for classification).
- **Task-relevant focus:** Choosing task-important tokens as global ensures the most important long-range relationships (question–answer, `[CLS]`–document) are modelled directly.

(d) Limitation of the proposed design

Information bottleneck for distant non-global tokens: Two ordinary tokens that are far apart (beyond the window) cannot attend to each other directly. Their interaction must pass through the few global tokens or through many layers of local windows, so the relationship may be diluted or lost. Since the pattern is fixed in advance, it may miss an important long-range dependency (e.g. a pronoun referring to an entity 2000 tokens earlier) that dense attention would capture directly.

Other limitations: selecting global tokens needs task-specific design; sparse patterns are harder to implement efficiently on GPUs (need custom kernels); for short sequences the overhead can make them no faster than dense attention.

Q6. Masked Self-Attention

What is masked self-attention? Why must future tokens be masked in autoregressive generation? Sequence: "The student is reading a book." (a) Which tokens are accessible when predicting the token after "reading"? (b) How an autoregressive Transformer generates one token at a time. (c) Masked self-attention in GPT vs bidirectional self-attention in BERT.

Part 1: Masked (causal) self-attention

Definition: Masked self-attention is self-attention in which each position i is allowed to attend only to positions $j \le i$ (itself and earlier tokens). Attention to future positions $j > i$ is blocked by adding a mask matrix M before the softmax:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \qquad M_{ij} = \begin{cases} 0 & \text{if } j \le i \\ -\infty & \text{if } j > i \end{cases}$$

Since $e^{-\infty} = 0$, the softmax weights for all future tokens become exactly 0, while the weights over allowed tokens still sum to 1. M is an upper-triangular matrix of $-\infty$ (the attention matrix is lower-triangular).

Why future tokens must be masked:

1. **Autoregressive factorization:** A language model predicts $P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$. The prediction of x_t must depend only on earlier tokens.
2. **Prevent cheating in training:** During training the whole target sentence is input at once (teacher forcing) for parallel computation. Without the mask, the position predicting x_{t+1} could simply attend to x_{t+1} itself and copy it; the loss would be near zero but the model would learn nothing useful.
3. **Train–inference consistency:** At inference the future tokens do not exist yet. The mask makes training conditions identical to generation conditions.
4. **Parallel training:** The mask lets all T next-token predictions be computed in one forward pass while keeping each prediction causal.

Part 2: Application to "The student is reading a book."

Tokens (one token per word and the full stop): $x_1 =$ The, $x_2 =$ student, $x_3 =$ is, $x_4 =$ reading, $x_5 =$ a, $x_6 =$ book, $x_7 =$ ".".

Causal attention mask (✓ = can attend, × = masked, $-\infty$):

Query ↓ / Key →	The	student	is	reading	a	book	.
The	✓	×	×	×	×	×	×
student	✓	✓	×	×	×	×	×
is	✓	✓	✓	×	×	×	×
reading	✓	✓	✓	✓	×	×	×
a	✓	✓	✓	✓	✓	×	×
book	✓	✓	✓	✓	✓	✓	×
.	✓	✓	✓	✓	✓	✓	✓

(a) Tokens accessible when predicting the next token after "reading"

The prediction of x_5 is made from the output at position 4 ("reading"), which, under causal masking, can attend only to positions 1 to 4:

$$P(x_5 \mid x_1, x_2, x_3, x_4) = P(x_5 \mid \text{The, student, is, reading})$$

Accessible: "The", "student", "is", "reading" (the current token and all previous tokens). **Masked (not accessible):** "a", "book", "." (future tokens). The model must predict "a" without seeing it.

For example, attention weights of the "reading" row might be $[0.10, 0.45, 0.15, 0.30, 0, 0, 0]$: the three future weights are exactly 0, and the four visible weights sum to 1.

(b) How an autoregressive Transformer generates a sequence one token at a time

1. **Input prompt:** Tokenize the prompt, e.g. "The student", and add positional information.
2. **Forward pass:** Pass the tokens through the stack of decoder layers (masked self-attention + FFN). Take the final hidden state of the *last* position.
3. **Next-token distribution:** Project it to vocabulary size and apply softmax: $P(x_{t+1} \mid x_{\leq t}) = \text{softmax}(h_t W_{\text{vocab}})$.
4. **Select a token:** by greedy decoding (most probable token), beam search, or sampling (temperature, top-k, top-p). Suppose "is" is chosen.
5. **Append and repeat:** The chosen token is appended to the input: "The student is", and steps 2–4 are repeated.
6. **Stop:** when an end-of-sequence token is generated or a maximum length is reached.

Example trace:

Step	Input (context)	Generated token
1	The student	is
2	The student is	reading
3	The student is reading	a
4	The student is reading a	book
5	The student is reading a book	.
6	The student is reading a book .	`<EOS>`

KV caching: the keys and values of earlier tokens do not change (because of the causal mask), so they are stored and reused; at each step only the new token's q, k, v are computed. Training is parallel (teacher forcing), but generation is inherently sequential.

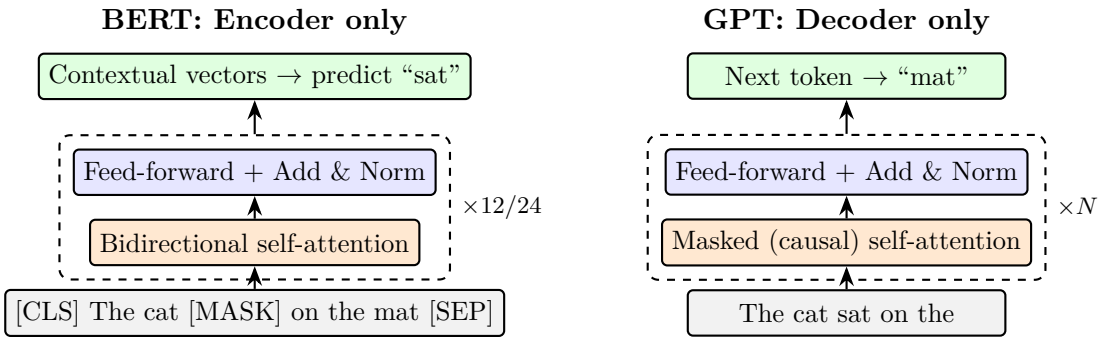
(c) Masked self-attention (GPT) vs bidirectional self-attention (BERT)

Aspect	Masked self-attention (GPT)	Bidirectional self-attention (BERT)
Visibility	Token i sees only tokens $1, \ldots, i$ (left context)	Token i sees all tokens $1, \ldots, n$ (left and right context)
Mask	Upper-triangular $-\infty$ mask; lower-triangular attention matrix	No causal mask (only padding mask); full $n \times n$ attention matrix
Model	Decoder-only Transformer	Encoder-only Transformer
Training objective	Next-token prediction (causal LM)	Masked language modelling (+ NSP)
Example ("reading")	Representation of "reading" uses only "The student is reading"	Representation of "reading" also uses "a book", so it knows what is being read
Strength	Natural text generation, one token at a time	Rich contextual understanding of every token
Weakness	Cannot use right-hand context when encoding a token	Cannot generate text autoregressively (would see the answer)
Typical tasks	Chatbots, story writing, code generation	Classification, NER, extractive question answering

Q7. BERT vs GPT

Compare BERT and GPT architectures in terms of encoder and decoder components. Explain their training objectives. Why is BERT suitable for understanding tasks and GPT for autoregressive generation? Give two applications each.

Part 1: Architecture



Aspect	BERT	GPT
Transformer part used	**Encoder only** (stack of encoder layers)	**Decoder only** (stack of decoder layers without the cross-attention sub-layer, since there is no encoder)
Sub-layers per block	Multi-head self-attention (bidirectional) + FFN, with residual and LayerNorm	Masked multi-head self-attention + FFN, with residual and LayerNorm
Attention direction	Bidirectional: each token sees both left and right context	Unidirectional (left-to-right): each token sees only previous tokens
Input format	`[CLS]` sentence A `[SEP]` sentence B `[SEP]`; token + segment + position embeddings	Plain token sequence; token + position embeddings
Output	Contextual embedding for every token; `[CLS]` vector for sentence-level tasks	Probability distribution over the next token
Sizes	Base: 12 layers, 768 hidden, 12 heads, 110M; Large: 24 layers, 1024 hidden, 16 heads, 340M	GPT-1: 117M; GPT-2: 1.5B; GPT-3: 175B

Part 2: Training objectives

BERT:

1. **Masked Language Modelling (MLM):** 15% of tokens are chosen; of these, 80% are replaced with `[MASK]`, 10% with a random token and 10% left unchanged. The model predicts the original token from both sides of the context:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \mathcal{M}} \log P(x_i \mid \tilde{x}),$$

   e.g. "The cat `[MASK]` on the mat" → predict "sat" using "The cat" *and* "on the mat".

2. **Next Sentence Prediction (NSP):** Given sentences A and B, predict from the `[CLS]` vector whether B actually follows A (50% real, 50% random). This trains sentence-pair understanding.

GPT:

- **Causal Language Modelling (next-token prediction):**

$$\mathcal{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log P(x_t \mid x_1, \ldots, x_{t-1}),$$

  e.g. "The cat sat on the" → predict "mat". Every token in the sequence provides a training signal.

Part 3: Why BERT suits understanding and GPT suits generation

BERT for language understanding:

- Understanding requires the meaning of each word in its *full* context. Bidirectional attention lets each token's representation use both left and right context (e.g. "bank" in "river bank" vs "bank account" is resolved by words on either side).
- The `[CLS]` representation summarizes the whole input for classification; token outputs are directly usable for tagging and span extraction.
- MLM trains exactly this skill: filling in a word using the whole sentence.
- But BERT is not designed to generate: it does not model $P(x_t \mid x_{<t})$, and with bidirectional attention it would see the tokens it is supposed to predict.

GPT for autoregressive generation:

- Generation produces text left-to-right, one token at a time. GPT's causal mask and next-token objective exactly match this process: training and inference behave the same way.
- The model gives a full probability distribution over the next token, which can be sampled repeatedly to produce fluent, open-ended text of any length.
- At large scale it supports zero/few-shot prompting, so many tasks can be done by generating the answer as text.
- But for pure understanding tasks, each token's representation sees only left context, which is less rich than BERT's bidirectional representation at the same size.

Part 4: Applications

BERT:

1. **Sentiment analysis / text classification** – e.g. classifying product reviews as positive or negative, spam detection (using the `[CLS]` vector).
2. **Extractive question answering and named entity recognition** – e.g. finding the answer span in a passage (SQuAD), tagging names of persons, places and organizations; also semantic search (Google Search uses BERT to understand queries).

GPT:

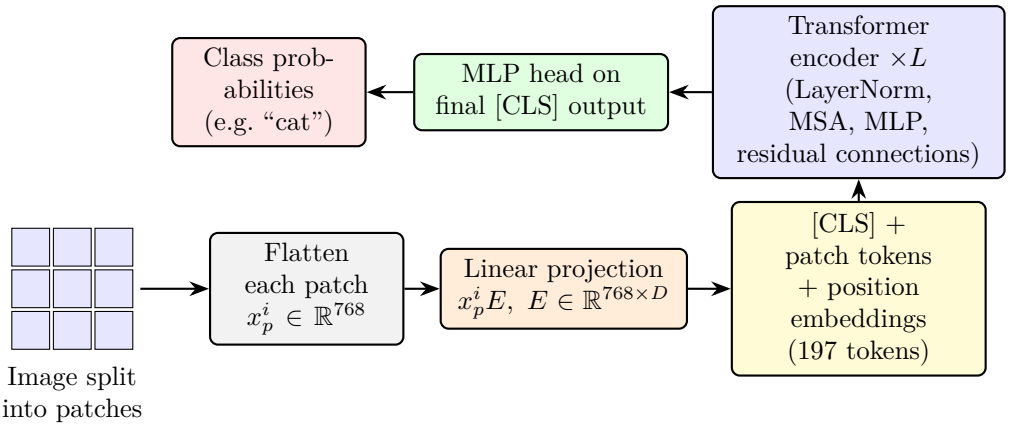
1. **Conversational chatbots / virtual assistants** – e.g. ChatGPT answering questions and holding dialogue.
2. **Content and code generation** – e.g. writing emails, stories and summaries; generating code from natural-language descriptions (GitHub Copilot).

Q8. Vision Transformer (ViT)

Explain the basic architecture of ViT. How are image patches converted into tokens? An RGB image of 224×224 is divided into non-overlapping 16×16 patches; calculate the number of patches. Explain the role of positional information and the encoder in ViT classification.

Part 1: Basic architecture of ViT (Dosovitskiy et al., 2020)

ViT applies a standard Transformer *encoder* directly to an image by treating the image as a sequence of patches – "An image is worth 16×16 words".



Components:

1. **Patch embedding:** The image is split into fixed-size patches, each flattened and linearly projected to a D-dimensional vector ("patch token").
2. **Class token `[CLS]`:** A learnable vector x_{class} is prepended to the patch tokens; its final state represents the whole image.
3. **Position embeddings:** Learnable 1-D position embeddings are added to all tokens.
4. **Transformer encoder:** L identical layers (ViT-Base: $L = 12$, $D = 768$, 12 heads, 86M parameters), each with pre-LayerNorm, multi-head self-attention (MSA), and an MLP with GELU, plus residual connections.
5. **Classification head:** An MLP (a single linear layer when fine-tuning) maps the final `[CLS]` output to class scores.

Equations:

$$
\begin{aligned}
z_0 &= [\,x_{\text{class}};\ x_p^1 E;\ x_p^2 E;\ \ldots;\ x_p^N E\,] + E_{\text{pos}}, && E \in \mathbb{R}^{(P^2C)\times D},\ E_{\text{pos}} \in \mathbb{R}^{(N+1)\times D} \\
z'_\ell &= \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1}, && \ell = 1, \ldots, L \\
z_\ell &= \text{MLP}(\text{LN}(z'_\ell)) + z'_\ell, && \ell = 1, \ldots, L \\
y &= \text{LN}(z_L^0) && (z_L^0 = \text{final } \texttt{[CLS]} \text{ state})
\end{aligned}
$$

Part 2: Converting image patches into tokens

For an image $x \in \mathbb{R}^{H\times W\times C}$ and patch size $P \times P$:

1. **Split** the image into $N = \dfrac{H \cdot W}{P^2}$ non-overlapping patches, each of size $P \times P \times C$.

2. **Flatten** each patch into a vector $x_p^i \in \mathbb{R}^{P^2C}$. For $P = 16$, $C = 3$: $16 \times 16 \times 3 = 768$ values.
3. **Linearly project** each flattened patch with a trainable matrix $E \in \mathbb{R}^{P^2C \times D}$ to obtain a D-dimensional patch embedding. (Equivalent to a convolution with kernel size 16 and stride 16.)
4. **Prepend** the learnable `[CLS]` token and **add** position embeddings. The result, a sequence of $N + 1$ vectors of size D, is processed exactly like a sequence of word tokens.

Part 3: Numerical – number of patches

Given: $H = W = 224$, $C = 3$ (RGB), patch size $P = 16$ (non-overlapping).
Formula: $N = \frac{H}{P} \times \frac{W}{P} = \frac{HW}{P^2}$.

$$\text{Patches along height} = \frac{224}{16} = 14$$
$$\text{Patches along width} = \frac{224}{16} = 14$$
$$N = 14 \times 14 = 196$$
$$\text{Check: } N = \frac{224 \times 224}{16 \times 16} = \frac{50{,}176}{256} = 196 \checkmark$$

Total number of patches $N = 196$.
Each patch has $16 \times 16 \times 3 = 768$ values. With the `[CLS]` token, the Transformer input sequence has $196 + 1 = 197$ tokens. (Check: $196 \times 768 = 150{,}528 = 224 \times 224 \times 3$ pixel values, so no pixel is lost.)

Part 4: Role of positional information and the encoder

Positional information:

- Self-attention is permutation-invariant, so without positions, the 196 patches would be an unordered set – the model could not distinguish an image from a shuffled version of its patches, and spatial structure (sky above ground, eyes above mouth) would be lost.
- ViT adds learnable position embeddings $E_{\text{pos}} \in \mathbb{R}^{197 \times D}$ to the patch tokens. During training these learn the 2-D layout: embeddings of patches in the same row or column become similar, and nearby patches have similar embeddings.
- When the image resolution changes at fine-tuning time, the position embeddings are 2-D interpolated to the new grid.

Encoder:

- Multi-head self-attention lets every patch attend to every other patch from the *first* layer, giving a global receptive field (unlike a CNN, which sees only a local region in early layers). This lets it relate distant parts of an image (e.g. a dog's head and tail).
- MLP layers transform the features of each patch; stacked layers build increasingly abstract features (edges $\rightarrow$ parts $\rightarrow$ objects).
- The `[CLS]` token attends to all patches in every layer, gathering the information needed for classification; its final state y is passed to the MLP head, and softmax gives the class probabilities. Training uses cross-entropy loss.
- ViT has weaker built-in image biases (locality, translation equivariance) than CNNs, so it needs large-scale pre-training (e.g. ImageNet-21k, JFT-300M) to outperform CNNs, after which it transfers very well.

Q9. Embedding and Head Dimensions for a Sentence

Sentence: "Deep learning improves image recognition." Number of tokens $n = 5$; embedding dimension $d = 128$. (a) Embedding vector dimension per token. (b) Input embedding matrix dimension. (c) Dimension after adding positional encoding of dimension 128. (d) Dimension of each head with 4 heads. (e) Why positional encoding and token embedding must have compatible dimensions.

Given: Tokens: [Deep, learning, improves, image, recognition] $\Rightarrow n = 5$; $d_{\text{model}} = 128$; $h = 4$.

(a) Dimension of the embedding vector for each token

Each token is mapped (by a row lookup in the embedding matrix $E \in \mathbb{R}^{|V| \times 128}$) to a vector of size d_{model}.

Each token embedding $e_i \in \mathbb{R}^{128}$, i.e. a 1×128 vector.

(b) Dimension of the complete input embedding matrix

Stacking the 5 token vectors as rows:

$$X = \begin{bmatrix} e_{\text{Deep}} \\ e_{\text{learning}} \\ e_{\text{improves}} \\ e_{\text{image}} \\ e_{\text{recognition}} \end{bmatrix} \in \mathbb{R}^{n \times d_{\text{model}}} = \mathbb{R}^{5 \times 128}$$

X is 5×128, containing $5 \times 128 = 640$ values.

(c) Dimension after adding positional encoding

The positional encoding matrix has one 128-dimensional vector per position: $PE \in \mathbb{R}^{5 \times 128}$. Element-wise addition does not change the shape:

$$Z = X + PE: \quad (5 \times 128) + (5 \times 128) = 5 \times 128$$

The resulting matrix Z is 5×128.

(d) Dimension of each attention head

$$d_k = d_v = \frac{d_{\text{model}}}{h} = \frac{128}{4} = 32$$

Each head has dimension 32. Per head: $Q_i, K_i, V_i \in \mathbb{R}^{5 \times 32}$ (using $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{128 \times 32}$); score matrix 5×5; head output 5×32. Concatenating 4 heads gives $5 \times (4 \times 32) = 5 \times 128$.

(e) Why the dimensions must be compatible

1. **Element-wise addition is defined only for equal shapes:** $z_{t,k} = x_{t,k} + p_{t,k}$ for every $k = 1, \ldots, 128$. Each dimension of the token embedding needs exactly one matching positional value; a 128-dimensional embedding cannot be added to, say, a 64-dimensional encoding.
2. **Every dimension receives position information:** The positional signal is spread across all 128 dimensions (different frequencies in sinusoidal encoding), so each feature can carry both meaning and position.
3. **Keeps the model dimension constant:** The sum stays 5×128, which is exactly what the first layer expects. The residual connections ($x + \text{Sublayer}(x)$) throughout the Transformer also require every layer's input and output to be $d_{\text{model}} = 128$.

4. **Alternative (concatenation)** would change the dimension (e.g. $128 + 128 = 256$) and increase the parameters of all following layers; addition avoids this.

Q10. Dimensions in an Encoder–Decoder Transformer

Encoder–decoder Transformer for machine translation: source length $n_s = 5$, target length $n_t = 3$, $d_{\text{model}} = 512$, $h = 8$. (a) Dimension of each head. (b) Q, K, V in encoder self-attention. (c) Q, K, V in decoder cross-attention. (d) Attention score matrices for encoder self-attention and decoder cross-attention. (e) Why Q comes from the decoder while K, V come from the encoder in cross-attention.

Given: $n_s = 5$, $n_t = 3$, $d_{\text{model}} = 512$, $h = 8$.
Encoder input/output: $X_{\text{enc}} \in \mathbb{R}^{5\times512}$. Decoder hidden states: $X_{\text{dec}} \in \mathbb{R}^{3\times512}$.

(a) Dimension of each attention head

$$d_k = d_v = \frac{d_{\text{model}}}{h} = \frac{512}{8} = 64$$

Each head has dimension 64.

(b) Q, K, V in encoder self-attention

All three come from the encoder input X_{enc} (5×512):

$$\text{Full (all heads): } Q = X_{\text{enc}}W^Q,\ K = X_{\text{enc}}W^K,\ V = X_{\text{enc}}W^V: \quad (5\times512)(512\times512) = 5\times512$$

$$\text{Per head: } Q_i = X_{\text{enc}}W_i^Q: \quad (5\times512)(512\times64) = 5\times64 \quad (\text{same for } K_i, V_i)$$

Encoder self-attention: $Q, K, V \in \mathbb{R}^{5\times512}$ in total; for each head $Q_i, K_i, V_i \in \mathbb{R}^{5\times64}$.

(c) Q, K, V in decoder cross-attention

Query from the decoder (3 target tokens); keys and values from the encoder output (5 source tokens):

$$Q = X_{\text{dec}}W^Q : (3\times512)(512\times512) = 3\times512; \qquad Q_i : (3\times512)(512\times64) = 3\times64$$
$$K = X_{\text{enc}}W^K : (5\times512)(512\times512) = 5\times512; \qquad K_i : 5\times64$$
$$V = X_{\text{enc}}W^V : (5\times512)(512\times512) = 5\times512; \qquad V_i : 5\times64$$

Cross-attention: $Q \in \mathbb{R}^{3\times512}$ (per head 3×64); $K \in \mathbb{R}^{5\times512}$ (per head 5×64); $V \in \mathbb{R}^{5\times512}$ (per head 5×64).

(d) Attention score matrices

Encoder self-attention (per head):

$$Q_iK_i^\top : (5\times64)(64\times5) = 5\times5$$

All 8 heads: $8\times5\times5 = 200$ scores (tensor $8\times5\times5$). Output per head: $(5\times5)(5\times64) = 5\times64$.

Decoder cross-attention (per head):

$$Q_iK_i^\top : (3\times64)(64\times5) = 3\times5$$

All 8 heads: $8\times3\times5 = 120$ scores (tensor $8\times3\times5$). Output per head: $(3\times5)(5\times64) = 3\times64$; concatenated: 3×512.

Encoder self-attention score matrix: 5×5 per head (200 values over 8 heads).
Decoder cross-attention score matrix: 3×5 per head (120 values over 8 heads) – each of the 3 target positions has a weight for each of the 5 source tokens.

(For completeness, the decoder's masked self-attention score matrix is 3×3 per head, lower-triangular.)

Attention type	Q (per head)	K, V (per head)	Scores (per head)	Scores (8 heads)
Encoder self-attention	5×64	5×64	5×5	200
Decoder masked self-attention	3×64	3×64	3×3	72
Decoder cross-attention	3×64	5×64	3×5	120

(e) Why Query comes from the decoder and Key/Value come from the encoder

1. **Role of each:** The Query represents "what information is needed now"; Keys are used to find matching information; Values are the information returned.
2. **The decoder is the one asking:** At each target position, the decoder knows what it has generated so far and needs to know which source words are relevant to produce the next target word. So its states form the queries.
3. **The encoder holds the answer:** The meaning of the source sentence is stored in the encoder output. Keys (from the encoder) let each source token be matched against the decoder's query, and Values (from the encoder) carry the source content that is copied into the decoder.
4. **Alignment:** The 3×5 weight matrix is a soft alignment between target and source words, e.g. when generating the French word "chat", the query attends most to the English key "cat". This replaces the fixed context vector of RNN seq2seq models (Q1).
5. **Shape consistency:** Output $= \text{softmax}(Q_i K_i^\top / \sqrt{64}) V_i$ has shape $(3 \times 5)(5 \times 64) = 3 \times 64$ – one vector per *target* position, which is exactly what the decoder needs for its next sub-layer. If Q came from the encoder, the output would have 5 rows (source length) and would not align with the target positions.
6. **Same source context for every step:** The encoder output is computed once and reused as K and V for all decoding steps and all decoder layers, which is efficient.

End of solutions